



**UNIVERSIDAD
SERGIO ARBOLEDA**

Taller 1 corte 2

Lenguajes de programación

**Yslen Natalia Moreno Gutierrez
Santiago Patiño Sarmiento**

*Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 09 de septiembre de 2026*

1. Introducción

Se implementaron en ANTLR 4.13.2, con Python como lenguaje objetivo, las tres gramáticas propuestas en clase, y se analizaron los árboles que cada una produce. El trabajo cubre tres puntos: (1) la gramática clásica de expresiones de la diapositiva 11, con pruebas de evaluación; (2) las distintas formas del árbol de análisis para la gramática de la diapositiva 12; y (3) la gramática de la diapositiva 15, para comprobar si es ambigua.

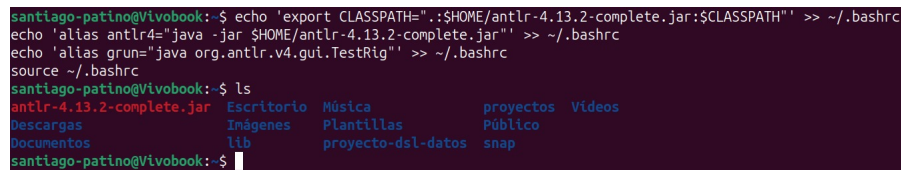
La evaluación numérica se resolvió en todos los casos con el patrón **Visitor**, generado por ANTLR con la opción `-visitor`.

Entorno de trabajo

Sobre Ubuntu se configuró el `CLASSPATH` y los alias de ANTLR, y se instaló el *runtime* de Python en la misma versión del generador (4.13.2); si las versiones no coinciden, el *parser* generado falla al importarse.

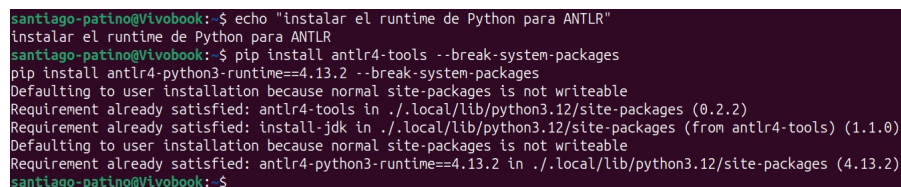
```
echo 'export CLASSPATH=".:$HOME/antlr-4.13.2-complete.jar:$CLASSPATH"' >> ~/.bashrc
echo 'alias antlr4="java -jar $HOME/antlr-4.13.2-complete.jar"' >> ~/.bashrc
echo 'alias grun="java org.antlr.v4.gui.TestRig"' >> ~/.bashrc
pip install antlr4-tools antlr4-python3-runtime==4.13.2 --break-system-packages
```

Listing 1: Alias, CLASSPATH e instalación del runtime



```
santiago-patino@Vivobook:~$ echo 'export CLASSPATH=".:$HOME/antlr-4.13.2-complete.jar:$CLASSPATH"' >> ~/.bashrc
santiago-patino@Vivobook:~$ echo 'alias antlr4="java -jar $HOME/antlr-4.13.2-complete.jar"' >> ~/.bashrc
santiago-patino@Vivobook:~$ echo 'alias grun="java org.antlr.v4.gui.TestRig"' >> ~/.bashrc
santiago-patino@Vivobook:~$ source ~/.bashrc
santiago-patino@Vivobook:~$ ls
antlr-4.13.2-complete.jar  Escritorio  Música  proyectos  Videos
Descargas                 Imágenes  Plantillas  Público
Documentos                lib        proyecto-dsl-datos  snap
santiago-patino@Vivobook:~$
```

Figura 1: Configurar Alias



```
santiago-patino@Vivobook:~$ echo "instalar el runtime de Python para ANTLR"
instalar el runtime de Python para ANTLR
santiago-patino@Vivobook:~$ pip install antlr4-python3-runtime==4.13.2 --break-system-packages
pip install antlr4-python3-runtime==4.13.2 --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: antlr4-tools in ./local/lib/python3.12/site-packages (0.2.2)
Requirement already satisfied: install-jdk in ./local/lib/python3.12/site-packages (from antlr4-tools) (1.1.0)
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: antlr4-python3-runtime==4.13.2 in ./local/lib/python3.12/site-packages (4.13.2)
santiago-patino@Vivobook:~$
```

Figura 2: Instalar el runtime de Python para ANTLR

2. Punto 1. Gramática de expresiones (diapositiva 11)

Gramática

La gramática `Expr.g4` reconoce expresiones con suma, multiplicación y paréntesis. La precedencia se fija con la estructura de la gramática: la suma se define en `e`, la multiplicación en `t` y los operandos en `f`, de modo que `*` queda más abajo en el árbol y se agrupa antes que `+`.

```

santiago-patino@Vivobook: ~/Escritorio/correccion/punto1
GNU nano 7.2                               Expr.g4 *
grammar Expr;

prog
: e EOF
;

e
: e '+' t # SumaExpr
| t # TerminoComoExpr
;

t
: t '*' f # Multiplicacion
| f # FactorComoTermino
;

f
: ID # FactorID
| NUM # FactorNum
| '(' e ')' # FactorParentesis
;

ID : [a-zA-Z_][a-zA-Z_0-9]*;
NUM : [0-9]+ ('.' [0-9]+)?;
WS : [ \t\r\n]+ -> skip;

```

Programa principal

El programa `main.py` construye el lexer y el parser generados por ANTLR, reemplaza el listener de errores por defecto por uno propio (`ContadorErrores`) que cuenta los errores sintácticos, e invoca la regla inicial `prog`. Al final decide si la cadena es aceptada o rechazada según el número de errores contados.

```

GNU nano 7.2                               main.py *
import sys
from antlr4 import InputStream, FileStream, CommonTokenStream
from antlr4.error.ErrorListener import ErrorListener
from ExprLexer import ExprLexer
from ExprParser import ExprParser

class ContadorErrores(ErrorListener):
    def __init__(self):
        super().__init__()
        self.errores = 0

    def syntaxError(self, recognizer, offendingSymbol, line, column, msg, e):
        self.errores += 1
        print(f"Error sintáctico en línea {line}, columna {column}: {msg}")

def procesar(entrada_stream, etiqueta):
    lexer = ExprLexer(entrada_stream)
    tokens = CommonTokenStream(lexer)
    parser = ExprParser(tokens)

    # Reemplazar el listener por defecto con nuestro contador
    parser.removeErrorListeners()
    listener = ContadorErrores()
    parser.addErrorListener(listener)

    parser.prog()

    if listener.errores == 0:
        print(f"ACEPTADA: '{etiqueta}'")
    else:
        print(f"RECHAZADA: '{etiqueta}' ({listener.errores} error(es))")

def main():
    if len(sys.argv) < 2:
        texto = input("Ingresa la expresión: ")
        entrada = InputStream(texto)
        procesar(entrada, texto)
    elif sys.argv[1] == '-f':
        if len(sys.argv) < 3:
            print(f"Error: falta la ruta del archivo tras -f")
            sys.exit(1)
        ruta = sys.argv[2]
        entrada = FileStream(ruta, encoding='utf-8')
        procesar(entrada, f"archivo:{ruta}")
    else:
        texto = sys.argv[1]
        entrada = InputStream(texto)
        procesar(entrada, texto)

if __name__ == '__main__':
    main()

```

Generación del parser

```
antlr4 -Dlanguage=Python3 Expr.g4
```

```
santiago-patino@Vivobook:~/Escritorio/correccion/punto1$ echo "Generar el parser en Python"
Generar el parser en Python
santiago-patino@Vivobook:~/Escritorio/correccion/punto1$ antlr4 -Dlanguage=Python3 Expr.g4
santiago-patino@Vivobook:~/Escritorio/correccion/punto1$
```

Figura 3: Generar parser

Criterio de aceptación

Una cadena es **ACEPTADA** cuando se cumplen las dos condiciones siguientes:

1. Todos sus caracteres corresponden a tokens definidos en la gramática: números (NUM), identificadores (ID), los operadores + y *, los paréntesis (y), y espacios en blanco.
2. La secuencia de tokens puede derivarse completa desde **prog**, es decir, forma una expresión bien construida que termina exactamente en EOF. En ese caso el contador del listener queda en 0 y el programa imprime **ACEPTADA**.

Una cadena es **RECHAZADA** cuando el parser reporta al menos un error, lo que ocurre en alguno de estos casos:

- Aparece un operador sin su operando, por ejemplo dos operadores seguidos (2+*4) o un operador al inicio (+8). En esa posición el parser esperaba el inicio de un factor: (, ID o NUM.
- Aparece un carácter que no pertenece al lenguaje, como -. El lexer lo descarta con un *token recognition error* y el parser recibe una secuencia de tokens que ya no encaja en la gramática.
- Los paréntesis no cierran correctamente, o sobran tokens después de una expresión completa (el parser esperaba EOF).

En estos casos el contador es mayor que 0 y el programa imprime **RECHAZADA** junto con el número de errores.

Casos aceptados

```
santiago-patino@Vivobook:~/Escritorio/correccion/punto1$ echo "Pruebas: Casos validos"
Pruebas: Casos validos
santiago-patino@Vivobook:~/Escritorio/correccion/punto1$ python3 main.py "2+3*4"
ACEPTADA: '2+3*4'
santiago-patino@Vivobook:~/Escritorio/correccion/punto1$ python3 main.py "2*3+4"
ACEPTADA: '2*3+4'
santiago-patino@Vivobook:~/Escritorio/correccion/punto1$ python3 main.py "1+2*3+4"
ACEPTADA: '1+2*3+4'
santiago-patino@Vivobook:~/Escritorio/correccion/punto1$ python3 main.py "7*8+2"
ACEPTADA: '7*8+2'
```

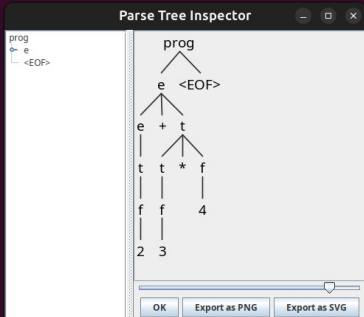
Casos rechazados

```
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "Prueba: Casos Invalidos"
Prueba: Casos Invalidos
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ python3 main.py "2+*4"
Error sintáctico en línea 1, columna 2: extraneous input '*' expecting {'(', ID, NUM}
RECHAZADA: '2+*4' (1 error(es))
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ python3 main.py "2+(3-7)"
line 1:4 token recognition error at: '-'
Error sintáctico en línea 1, columna 5: extraneous input '7' expecting ')'
RECHAZADA: '2+(3-7)' (1 error(es))
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ python3 main.py "+8"
Error sintáctico en línea 1, columna 0: extraneous input '+' expecting {'(', ID, NUM}
RECHAZADA: '+8' (1 error(es))
```

Árboles sintácticos con grun

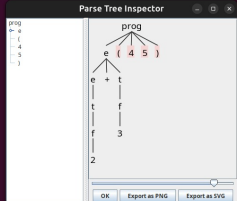
Cadena aceptada: $2 + 3 * 4$. El árbol se construye completo, sin nodos de error. La raíz prog deriva e <EOF>; e se divide en e + t, y el t de la derecha contiene $3 * 4$. Esto muestra que la multiplicación se agrupa primero.

```
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "Arbol grafico"
Arbol grafico
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ antlr4 Expr.g4
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ javac Expr*.java
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "2 + 3 * 4" | grun Expr prog -gui
```



Cadena rechazada: $2 + 3(4 - 5)$. Se reportan dos errores: *token recognition error at: '-'* en la columna 8, porque - no es un token válido, y *mismatched input '('* en la columna 5, porque después del número 3 el parser esperaba <EOF>, + o *. El árbol muestra en rojo los tokens que no pudieron ubicarse en la estructura.

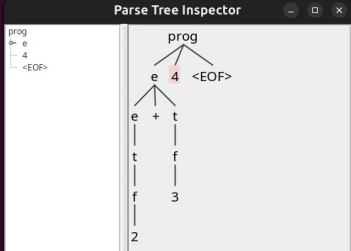
```
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "Arbol grafico"
Arbol grafico
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ antlr4 Expr.g4
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ javac Expr*.java
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "2 + 3 * 4" | grun Expr prog -gui
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "2 + 3(4 - 5)" | grun Expr prog -gui
line 1:8 token recognition error at: '-'
line 1:5 mismatched input '(' expecting [<EOF>, '+', '*']
```



Cadena rechazada: $2 + 3 - 4$. El lexer descarta el - (columna 6) y el parser recibe $2 + 3 4$. La expresión $2 + 3$ ya está completa, así que el 4 sobra: *extraneous input '4' expecting <EOF>*

en la columna 8.

```
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "Arbol grafico"
Arbol grafico
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ antlr4 Expr.g4
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ javac Expr*.java
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "2 + 3 * 4" | grun Expr prog -gui
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "2 + 3(4 - 5)" | grun Expr prog -gui
line 1:8 token recognition error at: '-'
line 1:5 mismatched input '(' expecting {<EOF>, '+', '*'}
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "2 + 3 - 4" | grun Expr prog -gui
line 1:6 token recognition error at: '-'
line 1:8 extraneous input '4' expecting <EOF>
```



The Parse Tree Inspector window shows a tree structure for the expression '2 + 3'. The root node is 'prog', which has a single child 'e'. Node 'e' has three children: 'e', '+', and 't'. Node 'e' (under '+') has a single child 't'. Node 't' (under '+') has a single child 'f'. Node 'f' (under '+') has a single child '2'. Node 't' (under '+') has a single child '3'.

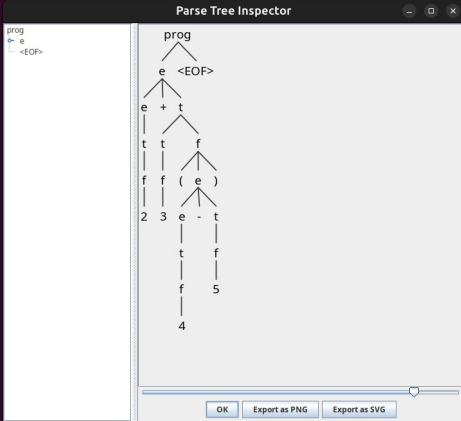
3. Punto 2. Formas del árbol (diapositiva 12)

ANTLR4 no construye un AST: construye siempre el **parse tree** concreto —con todos los nodos intermedios y todos los terminales, incluidos los paréntesis— y ofrece de él varias representaciones equivalentes. El AST se obtiene después, recorriendo ese árbol con un Visitor.

Tabla 1: Parse tree frente a AST

Parse tree (concreto)	AST (abstracto)
Un nodo por cada regla aplicada, incluidas las cadenas unitarias ($E \rightarrow T \rightarrow F \rightarrow num$).	Solo los nodos con contenido semántico; las cadenas unitarias desaparecen.
Conserva paréntesis y el operador como hoja.	Sin paréntesis; el operador pasa a ser el nodo padre.

```
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ echo "punto 2 : comprobar la s diferentes formas de AST"
punto 2 : comprobar las diferentes formas de AST
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ python3 main.py "2+3(4-5)"
'2+3(4-5)'
Arbol : (prog (e (e (t (f 2))) + (t (t (f 3)) (f ( (e (e (t (f 4))) - (t (f 5)) ) ) ) ) ) ) ) <EOF>)
Valor : -1
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ echo "2+3(4-5)" | grun Expr prog -gui
prog
-e
-EOF
```



The Parse Tree Inspector window shows a full parse tree for the expression '2+3(4-5)'. The root node is 'prog', which has a single child 'e'. Node 'e' has three children: 'e', '+', and 't'. Node 'e' (under '+') has a single child 't'. Node 't' (under '+') has a single child 'f'. Node 'f' (under '+') has a single child '2'. Node 't' (under '+') has a single child '3'. Node 't' (under '+') has a single child 'f'. Node 'f' (under '+') has a single child 'e'. Node 'e' (under 'f') has three children: 'e', '-', and 't'. Node 'e' (under 'f') has a single child 't'. Node 't' (under 'f') has a single child 'f'. Node 'f' (under 'f') has a single child '5'. Node 't' (under 'f') has a single child '4'.

3.1. Las dos formas comprobadas

Forma textual (tipo LISP). `toStringTree(recog=parser)` devuelve el árbol entre paréntesis; es la forma cómoda para documentar y comparar.

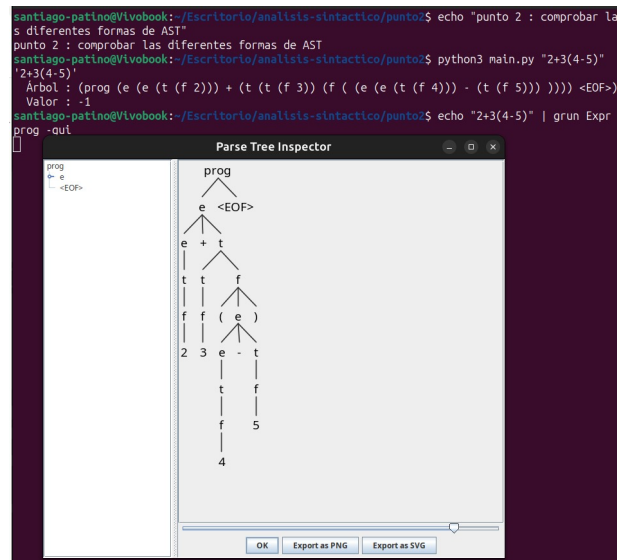
Forma gráfica. `grun` con la opción `-gui` abre el *Parse Tree Inspector*, que además exporta a PNG o SVG. Requiere haber generado también el destino Java, pero la gramática es la misma, así que el árbol mostrado es el mismo que produce el analizador de Python.

```
python3 main.py "2+3(4-5)"
echo "2+3(4-5)" | grun Expr prog -gui
```

Listing 2: Las dos representaciones del mismo árbol

Caso A: 2+3(4-5)

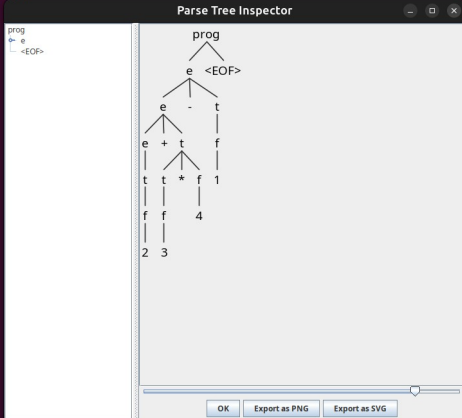
```
(prog (e (e (t (f 2))) + (t (t (f 3)) (f ( (e (e (t (f 4))) - (t (f 5))) )))) <EOF>)
Valor : -1
```



Caso B: 2+3*4-1

```
(prog (e (e (e (t (f 2))) + (t (t (f 3)) * (f 4))) - (t (f 1))) <EOF>)
Valor : 13
```

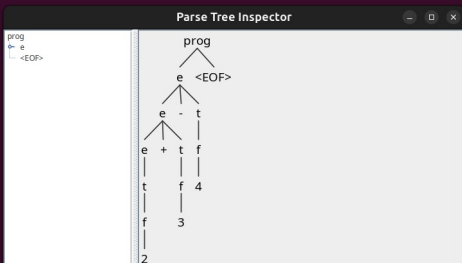
```
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ echo "punto 2 : comprobar las diferentes formas de AST"
punto 2 : comprobar las diferentes formas de AST
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ python3 main.py "2+3*4-1"
Árbol : (prog (e (e (e (t (f 2))) + (t (t (f 3))) * (f 4))) - (t (f 1))) <EOF>)
Valor : 13
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ echo "2+3*4-1" | grun Expr prog -out
```



Caso C: 2+3-4

```
(prog (e (e (e (t (f 2))) + (t (f 3))) - (t (f 4))) <EOF>)
Valor : 1
```

```
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ echo "punto 2 : comprobar las diferentes formas de AST"
punto 2 : comprobar las diferentes formas de AST
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ python3 main.py "2+3-4"
Árbol : (prog (e (e (e (t (f 2))) + (t (f 3))) - (t (f 4))) <EOF>)
Valor : 1
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ echo "2+3-4" | grun Expr prog -out
```



3.2. Análisis

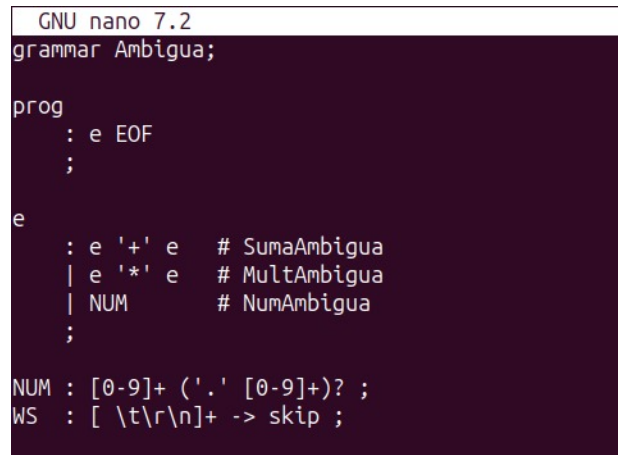
Las dos formas son la misma estructura: cada paréntesis de la notación LISP corresponde a un nivel de la ventana gráfica. En el caso A la forma gráfica muestra lo que la textual deja implícito —los paréntesis aparecen como **hojas** del nodo **f**—, y son justamente esas hojas las que un AST eliminaría, porque la agrupación ya está dada por la forma del árbol.

La distancia entre ambos árboles se mide en nodos vacíos: para $2+3-4$, el parse tree usa la cadena $e \rightarrow t \rightarrow f \rightarrow \text{NUM}$ para llegar a cada número, mientras que el AST equivalente tendría cinco nodos en total. El Visitor es precisamente el paso de uno al otro: al escribir `visitTerminoComoExpr` como `return self.visit(ctx.t())` se colapsa una cadena unitaria, y al ignorar los paréntesis en `visitFactorParentesis` se descartan terminales sin contenido semántico.

4. Punto 3. Gramática ambigua (diapositiva 15)

Gramática ambigua

La gramática `Ambigua.g4` define la suma y la multiplicación en una sola regla, sin separar niveles de precedencia:



```
GNU nano 7.2
grammar Ambigua;

prog
: e EOF
;

e
: e '+' e # SumaAmbigua
| e '*' e # MultAmbigua
| NUM    # NumAmbigua
;

NUM : [0-9]+ ('.' [0-9]+)? ;
WS  : [ \t\r\n]+ -> skip ;
```

4.1. Análisis

La gramática es ambigua porque una misma cadena admite más de un árbol de derivación. Por ejemplo, $2 + 3 * 4$ puede derivarse como $(2 + 3) * 4$ o como $2 + (3 * 4)$, y en ambos casos se usan solo las reglas de `e`.

Cómo resuelve ANTLR la ambigüedad

ANTLR 4 sí acepta y ejecuta gramáticas con recursión izquierda directa como esta. Al generar el parser, reescribe la regla `e` y le asigna a cada alternativa un nivel de precedencia según el orden en que aparece: **la alternativa escrita primero tiene mayor precedencia**. Además, los operadores binarios son asociativos por la izquierda por defecto.

Como la ambigüedad queda resuelta de forma fija con ese orden de alternativas, **ANTLR 4 no muestra ninguna advertencia** al generar el parser de esta gramática; por eso el comando `antlr4` termina sin mensajes. Que no aparezca un *warning* no significa que la gramática no sea ambigua: significa que ANTLR eligió una de las interpretaciones posibles sin avisar.

En `Ambigua.g4` la suma está escrita antes que la multiplicación, así que `+` queda con **mayor** precedencia que `*`. Esto es lo contrario de la convención matemática, y se ve en los árboles generados.

Generación del parser y pruebas

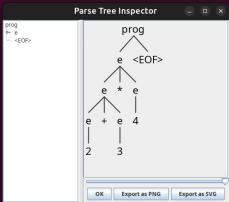
```
antlr4 -Dlanguage=Python3 Ambigua.g4
antlr4 Ambigua.g4
javac Ambigua*.java
```

Las tres primeras cadenas se aceptan: la ambigüedad no produce errores, porque ANTLR siempre escoge una derivación. Las dos últimas se rechazan porque no existe *ninguna* derivación posible. Esa es la diferencia entre una cadena ambigua y una cadena inválida.

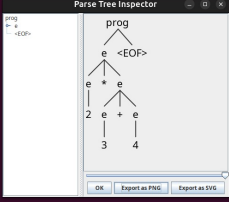
```
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ echo "Generar el parser en Python"
Generar el parser en Python
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ antlr4 -Dlanguage=Python3 Ambigua.g4
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ echo "Generar también en Java (para grun)"
Generar también en Java (para grun)
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ antlr4 Ambigua.g4
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ javac Ambigua*.java
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ nano main.py
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ echo "casos con operadores mezclados (donde hay ambigüedad real)"
casos con operadores mezclados (donde hay ambigüedad real)
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ python3 main.py "2 + 3 * 4"
ACEPTADA (ambigüedad resuelta por ANTLR): "2 + 3 * 4"
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ python3 main.py "2 * 3 + 4"
ACEPTADA (ambigüedad resuelta por ANTLR): "2 * 3 + 4"
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ python3 main.py "2 + 3 + 4"
ACEPTADA (ambigüedad resuelta por ANTLR): "2 + 3 + 4"
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ echo "Ejecutar caso inválido (para comparar rechazo real vs ambigüedad)"
Ejecutar caso inválido (para comparar rechazo real vs ambigüedad)
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ python3 main.py "2 + * 4"
Error sintáctico en línea 1, columna 4: extraneous input '*' expecting NUM
RECHAZADO: "2 + * 4" (1 error(s))
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ python3 main.py "2 * 3"
Error sintáctico en línea 1, columna 8: extraneous input '+' expecting NUM
RECHAZADO: "2 * 3" (1 error(s))
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$
```

Árboles sintácticos

```
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ echo "Visualizar el arbol con grun"
Visualizar el arbol con grun
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ echo "2 + 3 * 4" | grun Ambigua prog -gui
```



```
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ echo "Visualizar el arbol con grun"
Visualizar el arbol con grun
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ echo "2 * 3 + 4" | grun Ambigua prog -gui
santiago-patino@vivobook: ~/Escritorio/analisis-sintactico/punto5$ echo "2 * 3 + 4" | grun Ambigua prog -gui
```



2 + 3 * 4. La raíz de la expresión es $e * e$: el hijo izquierdo es $e + e$ con 2 y 3, y el derecho es 4. Es decir, se calcula $(2 + 3) * 4 = 20$ y no $2 + (3 * 4) = 14$.

2 * 3 + 4. La raíz es otra vez $e * e$: el hijo izquierdo es 2 y el derecho es $e + e$ con 3 y 4. Se calcula $2 * (3 + 4) = 14$ y no $(2 * 3) + 4 = 10$.

Conclusión

La gramática ambigua reconoce exactamente las mismas cadenas que una gramática con precedencia bien definida, pero no determina por sí sola cómo agruparlas. ANTLR 4 resuelve esa ambigüedad en silencio usando el orden de las alternativas, y en este caso produce agrupaciones distintas a las de la aritmética usual. Para obtener la precedencia correcta basta con escribir la alternativa de $*$ antes que la de $+$, o separar los niveles en reglas distintas como en la gramática del punto 1.